

UNIT-I: Fundamentals of Deep Networks

1. Introduction to Deep Learning

Deep Learning (DL) is a branch of Artificial Intelligence that enables computers to learn from large amounts of data using deep neural networks. It is designed to automatically learn patterns, features, and representations from raw data without requiring manual feature extraction.

Unlike traditional machine learning algorithms, deep learning models consist of multiple layers of interconnected artificial neurons. These layers process the input data step by step, allowing the model to learn simple as well as complex patterns. As the number of hidden layers increases, the model becomes capable of solving more sophisticated problems.

Deep Learning has gained significant importance due to the availability of large datasets, high-performance computing systems, and powerful Graphics Processing Units (GPUs). It has achieved remarkable success in solving complex real-world problems such as image recognition, speech recognition, natural language processing, medical diagnosis, and autonomous driving.

Definition

A subset of Machine Learning that uses multi-layered artificial neural networks to automatically learn hierarchical representations from large amounts of data.

Characteristics of Deep Learning

1. **Multiple Hidden Layers:** Deep Learning models consist of multiple hidden layers between the input and output layers, enabling them to learn complex relationships in data.
2. **Automatic Feature Learning:** Deep Learning automatically extracts relevant features from raw data, eliminating the need for manual feature engineering.
3. **Hierarchical Learning:** The lower layers learn simple features, while deeper layers combine them to recognize complex patterns and representations.
4. **Handles Large Datasets:** Deep Learning performs effectively when trained on large volumes of data.
5. **End-to-End Learning:** Deep Learning models learn directly from raw input data and produce the desired output without requiring multiple intermediate processing stages.
6. **High Prediction Accuracy:** Deep Learning provides high accuracy in solving complex problems involving images, speech, text, and videos.

Advantages of Deep Learning

- Automatically extracts meaningful features from raw data.
- Produces high accuracy for complex prediction and classification tasks.
- Efficiently handles structured as well as unstructured data.
- Learns complex relationships within large datasets.
- Reduces the need for manual feature engineering.
- Continuously improves performance with larger datasets.

Limitations of Deep Learning

- Requires a large amount of training data.
- Needs powerful computational resources such as GPUs.
- Training deep models requires considerable time.
- Many deep learning models are difficult to interpret and are often considered black-box models.
- Improper training may lead to overfitting.

Applications of Deep Learning

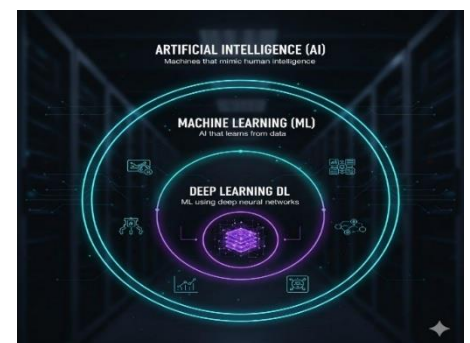
Deep Learning is widely used in various domains due to its ability to process complex and high-dimensional data. Some important applications include:

- **Computer Vision** – Image classification, object detection, and face recognition.
- **Speech Recognition** – Voice assistants and speech-to-text systems.
- **Natural Language Processing**–Machine translation, chatbots, sentiment analysis, text summarization.
- **Healthcare** – Disease diagnosis, medical image analysis, and drug discovery.
- **Autonomous Vehicles** – Lane detection, obstacle detection, and self-driving systems.
- **Recommendation Systems** – Personalized recommendations for movies, music, and online shopping.
- **Finance** – Fraud detection, credit risk assessment, and stock market prediction.
- **Robotics** – Industrial automation and intelligent robotic systems.

2. Artificial Intelligence (AI) vs Machine Learning (ML) vs Deep Learning (DL)

Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL) are related fields in computer science. AI is the broadest field that aims to build intelligent systems. Machine Learning is a subset of AI that enables computers to learn from data without explicit programming. Deep Learning is a subset of Machine Learning that uses multi-layer neural networks to automatically learn complex patterns from large amounts of data.

Artificial Intelligence $\supset$ Machine Learning $\supset$ Deep Learning



Difference between AI, ML, and DL:

Feature	Artificial Intelligence (AI)	Machine Learning (ML)	Deep Learning (DL)
Definition	Develops intelligent systems capable of performing tasks that require human intelligence.	Enables computers to learn from data without explicit programming.	Uses deep neural networks to automatically learn complex patterns from data.
Scope	Broadest field	Subset of AI	Subset of ML
Learning Method	Uses rules, logic, search techniques, and learning algorithms.	Learns from data using machine learning algorithms.	Learns using deep neural networks with multiple hidden layers.
Feature Extraction	Mostly manual	Manual or partially automatic	Fully automatic
Data Requirement	Moderate	Large	Very Large
Computational Requirement	Low to Moderate	Moderate	High
Performance	Suitable for general intelligent tasks	Good for prediction and classification problems	Excellent for complex problems involving images, speech, and text
Examples	Expert Systems, Robotics	Spam Detection, Recommendation Systems	Face Recognition, Autonomous Vehicles

3. Introduction to Neural Networks

A Neural Network, also known as an Artificial Neural Network (ANN), is a computational model inspired by the structure and functioning of the human brain. It consists of interconnected processing units called artificial neurons, which work together to process information, recognize patterns, and solve complex problems. A neural network learns from data by adjusting the connections between neurons during the training process. Once trained, it can make predictions, classify data, recognize patterns, and support decision-making. Neural networks form the foundation of modern Deep Learning and are widely used in applications such as image recognition, speech recognition, natural language processing, and medical diagnosis. Unlike traditional programming, where the programmer explicitly defines the rules, a neural network learns these rules automatically from the training data.

Definition

A network of interconnected artificial neurons that processes information by learning patterns from data through training.

Basic Components of a Neural Network

A neural network consists of the following basic components:

1. **Neurons:** A neuron is the basic processing unit of a neural network. Each neuron receives input, performs computations, and produces an output.
2. **Layers:** Neurons are organized into different layers. These layers work together to process the input data and generate the final output. The three basic layers are: Input layer, Hidden layers, Output layer.
3. **Connections:** Neurons in one layer are connected to neurons in the next layer. These connections allow information to flow through the network.
4. **Weights:** Each connection between neurons is assigned a numerical value called a **weight**. Weights determine the importance of each input during computation.
5. **Bias:** A bias is an additional parameter that helps the network adjust its output more effectively and improves learning capability.
6. **Activation Function:** The activation function determines whether a neuron should be activated and what output it should produce. It introduces non-linearity into the network, enabling it to learn complex patterns.

Working of a Neural Network

The basic working of a neural network involves the following steps:

1. The input layer receives the input data.
2. The data is passed to the hidden layer(s).
3. Each neuron performs mathematical computations using the input values, weights, and bias.
4. The activation function determines the output of each neuron.
5. The processed information is forwarded through successive layers.
6. The output layer generates the final prediction or classification.
7. During training, the network adjusts its weights and biases to improve prediction accuracy.

Applications of Neural Networks

1. **Image Recognition:** Identifying objects, faces, and handwritten characters in images.
2. **Speech Recognition:** Converting spoken language into text and enabling voice assistants.
3. **Natural Language Processing (NLP):** Used in language translation, chatbots, and text analysis.
4. **Medical Diagnosis:** Assisting in disease detection and medical image analysis.
5. **Recommendation Systems:** Providing personalized recommendations for movies, music, and online shopping.

4. Artificial Neuron

An Artificial Neuron, also called a processing element or node, is the basic building block of an Artificial Neural Network (ANN). It is a mathematical model inspired by the biological neuron. It receives one or more input values, processes them using weights, bias, and an activation function, and produces an output. Artificial neurons work together to form neural networks that can perform tasks such as classification, prediction, pattern recognition, and decision-making. They are the foundation of Artificial Neural Networks and Deep Learning models.

“An artificial neuron is a computational model that receives input signals, processes them using weights, bias, and an activation function, and produces an output”.

Components of an Artificial Neuron

An artificial neuron consists of the following components:

- 1. Inputs ($x_1, x_2, x_3, \dots, x_n$):** Inputs are the values or features provided to the neuron. They represent the information received from the external environment or from neurons in the previous layer.
- 2. Weights ($w_1, w_2, w_3, \dots, w_n$):** Each input is associated with a weight. A weight determines the importance or influence of the corresponding input on the neuron's output. During training, these weights are adjusted to improve the performance of the neural network.
- 3. Bias (b):** The bias is an additional parameter added to the weighted sum of inputs. It helps the neuron adjust its output and improves the flexibility of the learning process.
- 4. Summation Unit:** The summation unit computes the weighted sum of all input values along with the bias. The mathematical expression is:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

x_i = Input value

w_i = Weight associated with the input

b = Bias

z = Weighted sum

- 5. Activation Function:** The activation function receives the weighted sum as input and determines the final output of the neuron. It introduces non-linearity into the network, enabling it to learn complex patterns.
- 6. Output (y):** The output is the final result produced by the neuron after applying the activation function. This output is passed to the next neuron or becomes the final prediction.

Working of an Artificial Neuron

1. The neuron receives one or more input values.
2. Each input is multiplied by its corresponding weight.
3. The weighted inputs are added together.
4. A bias value is added to the weighted sum.
5. The resulting value is passed through an activation function.
6. The activation function produces the final output.
7. The output is transmitted to the next neuron or the output layer.

Advantages of an Artificial Neuron

- Learns from training data.
- Supports nonlinear decision-making through activation functions.
- Forms the basis for constructing complex neural networks.
- Suitable for pattern recognition and prediction tasks.

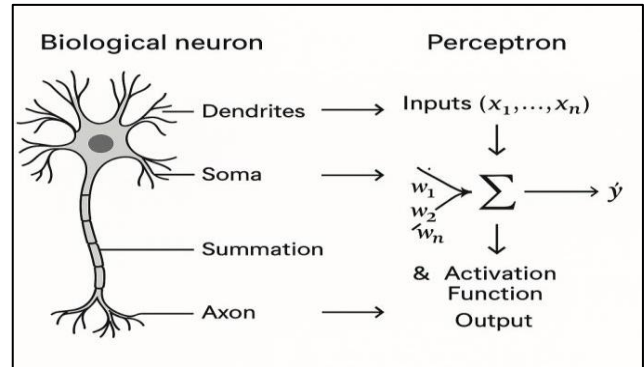
Limitations of an Artificial Neuron

- A single neuron can solve only simple problems.
- Cannot learn complex nonlinear relationships without hidden layers.
- Requires training to produce accurate predictions.

5. Biological Neuron vs Artificial Neuron

A biological neuron is the basic unit of the human nervous system, while an artificial neuron is its computational model used in Artificial Neural Networks (ANNs). Although artificial neurons are inspired by biological neurons, they differ in their structure, processing mechanism, and learning approach.

A biological neuron transmits information through electrochemical signals, whereas an artificial neuron processes numerical data using mathematical operations.



Difference between Biological Neuron and Artificial Neuron

Biological Neuron	Artificial Neuron
A biological neuron is a natural nerve cell found in the human nervous system.	An artificial neuron is a mathematical model used in artificial neural networks.
It receives and transmits information through electrochemical signals.	It receives and processes numerical input values.
Dendrites receive signals from other neurons.	Input nodes receive input data from external sources or previous neurons.
The cell body (soma) processes the received signals.	The summation unit computes the weighted sum of the inputs.
The axon carries signals to other neurons.	The output is forwarded to the next neuron or output layer.
Synapses connect one neuron to another.	Weights represent the strength of connections between neurons.
Learning occurs through changes in synaptic connections.	Learning occurs by adjusting weights and bias during training.
Signal transmission is relatively slow.	Mathematical computations are performed at very high speed.
Highly complex biological structure.	Comparatively simple computational model.
Used in the human brain and nervous system.	Used in Artificial Intelligence, Machine Learning, and Deep Learning models.

6. Evolution of Neural Networks and Deep Learning

Deep Learning has evolved through the continuous development of neural network architectures over several decades. Each new model was designed to overcome the limitations of earlier models and improve learning capability. The evolution began with the Perceptron and gradually advanced to sophisticated architectures such as Multi-Layer Perceptrons (MLPs), Restricted Boltzmann Machines (RBMs), Autoencoders, Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Transformers.

Evolution Timeline

Year	Model	Major Contribution
1958	Perceptron	Introduced the first single-layer neural network for binary classification.
1986	Multi-Layer Perceptron (MLP)	Introduced hidden layers and the backpropagation algorithm, enabling the learning of complex patterns.

2006	Restricted Boltzmann Machine (RBM)	Enabled unsupervised feature learning and became the foundation for Deep Belief Networks (DBNs).
2006–2008	Autoencoder	Learned efficient feature representations and performed dimensionality reduction.
2012 onwards	CNN, RNN, LSTM, Transformers	Specialized architectures for computer vision, sequential data, speech processing, and natural language processing.

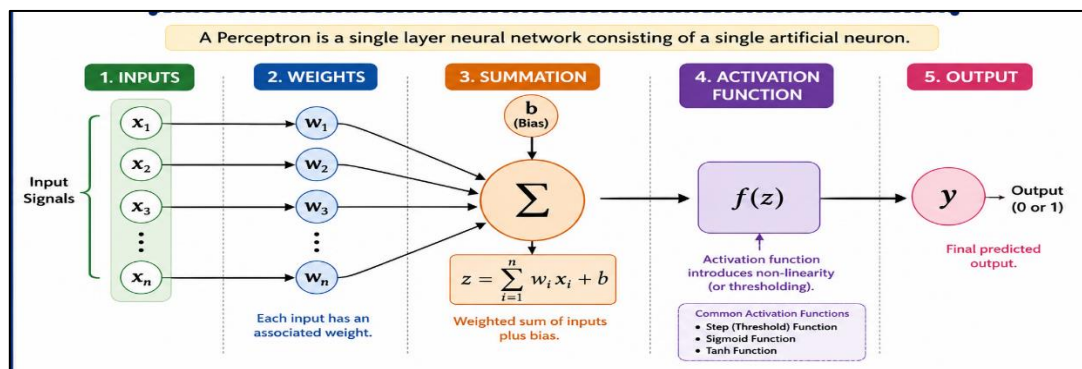
7. Perceptron

A Perceptron is the simplest form of an Artificial Neural Network (ANN) and the foundation of modern neural networks. It was introduced by Frank Rosenblatt in 1958. A perceptron consists of a single artificial neuron that receives multiple inputs, processes them using weights and bias, and produces a binary output. The perceptron is mainly used for binary classification problems, where the output belongs to one of two classes (e.g., Yes/No or 0/1). It learns by adjusting its weights during training to classify input data correctly. Although a perceptron can solve only linearly separable problems, it laid the foundation for the development of Multi-Layer Perceptrons and modern Deep Learning models.

Architecture of a Perceptron

A perceptron consists of the following components:

- **Input Layer:** Receives input features.
- **Weights:** Determine the importance of each input.
- **Bias:** Helps adjust the decision boundary.
- **Summation Unit:** Calculates the weighted sum of inputs.
- **Activation Function:** Produces the final binary output.
- **Output:** Represents the predicted class (0 or 1).



Working of a Perceptron

The working of a perceptron involves the following steps:

1. Receive the input values.
2. Multiply each input by its corresponding weight.
3. Add all weighted inputs and the bias.
4. Apply the activation (step) function.
5. Produce the final binary output.
6. During training, update the weights if the predicted output differs from the expected output.

Advantages of Perceptron

- Simple and easy to implement.
- Suitable for binary classification problems.
- Learns automatically by updating weights.
- Forms the foundation of modern neural networks.

Limitations of Perceptron

- Can solve only linearly separable problems.
- Cannot solve nonlinear problems such as the XOR problem.
- Uses only a single layer and produces binary outputs.

Applications of Perceptron

- Binary classification
- Pattern recognition
- Spam email detection
- Character recognition
- Simple decision-making systems

8. Multi-Layer Perceptron (MLP)

A Multi-Layer Perceptron (MLP) is an advanced form of the perceptron that consists of multiple layers of interconnected neurons. Unlike a single-layer perceptron, an MLP contains one or more hidden layers between the input and output layers, enabling it to learn complex and non-linear relationships in data. MLPs use the backpropagation algorithm for training, which adjusts the weights and biases to minimize prediction errors. Due to their ability to solve complex problems, MLPs are widely used in classification, regression, pattern recognition, and prediction tasks.

Definition

A Multi-Layer Perceptron (MLP) is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer that learns complex patterns using the backpropagation algorithm.

Architecture of a Multi-Layer Perceptron

1. Input Layer

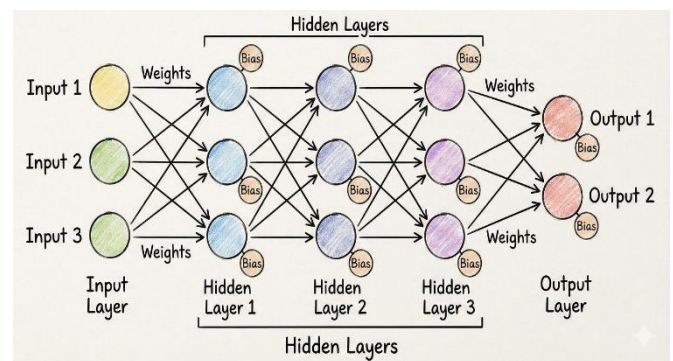
- Receives the input features from the dataset.
- Passes the data to the hidden layer.

2. Hidden Layer(s)

- One or more hidden layers process the input data.
- Each neuron performs computations using weights, bias, and an activation function.
- Hidden layers enable the network to learn complex patterns.

3. Output Layer

- Produces the final prediction or classification result.
- The number of output neurons depends on the application.



Working of a Multi-Layer Perceptron

1. Input data is provided to the input layer.
2. The input is forwarded to the hidden layer(s).
3. Each neuron computes the weighted sum of inputs and applies an activation function.
4. The processed information is passed from one layer to the next.
5. The output layer generates the final prediction.
6. During training, the backpropagation algorithm updates the weights and biases to reduce the prediction error.

Advantages of MLP

- Can solve both linear and non-linear problems.
- Learns complex patterns from data.
- Provides higher prediction accuracy than a single-layer perceptron.
- Widely used in classification and regression problems.

Limitations of MLP

- Requires more computational resources.
- Training can be time-consuming for large datasets.
- Performance depends on proper selection of network architecture and hyperparameters.

Applications of MLP

- Image classification
- Handwritten character recognition
- Speech recognition
- Medical diagnosis
- Pattern recognition

Difference between Perceptron and Multi-Layer Perceptron

Perceptron	Multi-Layer Perceptron (MLP)
Consists of a single layer.	Consists of multiple layers with one or more hidden layers.
Solves only linearly separable problems.	Solves both linear and non-linear problems.
Does not contain hidden layers.	Contains one or more hidden layers.
Uses a simple learning rule.	Uses the backpropagation algorithm for learning.
Lower learning capability.	Higher learning capability and accuracy.

9. Neural Network Architecture

A Neural Network Architecture refers to the arrangement of neurons and the connections between them in an Artificial Neural Network (ANN). It defines how input data flows through different layers of the network to produce the desired output.

A typical neural network consists of three types of layers: the Input Layer, Hidden Layer(s), and Output Layer. Each layer contains one or more neurons that work together to process information and solve complex problems.

The architecture of a neural network determines its learning capability, accuracy, and performance.

Components of a Neural Network Architecture**1. Input Layer**

- The first layer of the neural network.
- Receives the input features from the dataset.
- Passes the input data to the hidden layer.
- Does not perform complex computations.

2. Hidden Layer(s)

- Located between the input and output layers.
- Consists of one or more layers of neurons.
- Performs mathematical computations on the input data.
- Learns patterns and relationships from the data.
- A network may contain one or multiple hidden layers depending on the complexity of the problem.

3. Output Layer

- The final layer of the neural network.
- Produces the prediction or classification result.
- The number of output neurons depends on the application.

Information Flow in a Neural Network

The information in a neural network flows through the following steps:

1. Input data is received by the input layer.
2. The hidden layer(s) process the input using weights, bias, and activation functions.
3. The processed information is forwarded to the next layer.
4. The output layer produces the final prediction or classification.

Types of Neural Network Architecture

Based on the flow of information, neural networks can be classified into:

1. Feedforward Neural Network (FNN)

- Information flows only in one direction.
- No feedback or loops are present.
- Simple and widely used architecture.

2. Feedback (Recurrent) Neural Network

- Information can flow in both forward and backward directions through feedback connections.
- Suitable for sequential and time-dependent data.

Advantages of Neural Network Architecture

- Learns complex patterns from data.
- Supports parallel information processing.
- Can model both linear and non-linear relationships.
- Easily scalable by increasing the number of hidden layers and neurons.

Applications of Neural Network Architecture

- Image recognition
- Speech recognition
- Natural Language Processing
- Medical diagnosis
- Financial forecasting
- Recommendation systems

10. Training of Neural Networks

Training is the process of teaching a neural network to make accurate predictions by learning from a set of training data. During training, the network adjusts its weights and biases to minimize the difference between the predicted output and the actual output.

The objective of training is to enable the neural network to recognize patterns, learn relationships in data, and make accurate predictions for new, unseen inputs.

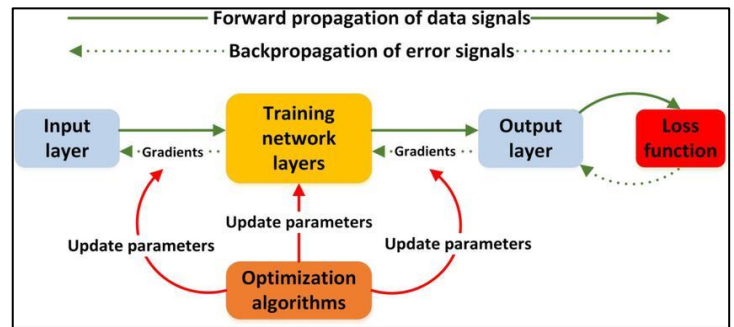
Need for Training

- Learn patterns from the training data.
- Improve prediction accuracy.
- Reduce errors in the output.
- Generalize knowledge to new data.
- Build an effective prediction model

Training Process

The training of a neural network generally involves the following steps:

1. **Initialize the Network:** Assign initial values to the weights and biases.
2. **Provide Training Data:** Input data is fed into the neural network.
3. **Generate Output:** The network processes the input and produces an output.
4. **Calculate Error:** The predicted output is compared with the actual output to determine the error.
5. **Update Weights and Biases:** The weights and biases are adjusted to reduce the error.
6. **Repeat the Process:** The above steps are repeated for multiple iterations until the error is minimized or the desired accuracy is achieved.



Important Terms in Training

Epoch: An Epoch is one complete pass of the entire training dataset through the neural network.

Batch: A Batch is a subset of the training dataset processed at one time during training.

Iteration: An Iteration is one update of the model's weights after processing a single batch.

Forward Propagation

Forward Propagation is the process by which input data passes through the neural network from the input layer to the output layer to generate a prediction. During this process, each neuron receives input values, performs computations using weights and bias, applies an activation function, and passes the output to the next layer.

Forward propagation is the first step in training and prediction. It calculates the output of the neural network based on the current values of weights and biases.

Definition

Forward Propagation is the process of passing input data through the neural network from the input layer to the output layer to compute the predicted output.

Steps in Forward Propagation

The forward propagation process involves the following steps:

1. Input data is provided to the input layer.
2. Each neuron calculates the weighted sum of its inputs.
3. The bias is added to the weighted sum.
4. The activation function is applied to produce the neuron's output.
5. The output is passed to the next layer.
6. The process continues until the output layer produces the final prediction.

Mathematical Representation

For each neuron, the weighted sum is calculated as:

$$z = \sum_{i=1}^n w_i x_i + b$$

The activation function then computes the output:

$$y = f(z)$$

where:

x_i = Input

w_i = Weight

b = Bias

z = Weighted sum

$f(z)$ = Activation function

y = Output

Advantages of Forward Propagation

- Produces the predicted output of the network.
- Forms the first step in the learning process.
- Enables information to flow through the network.
- Used during both training and testing.

Backpropagation

Backpropagation is a supervised learning algorithm used to train multi-layer neural networks. It works by calculating the error between the predicted output and the actual output, and then adjusting the weights and biases to minimize this error.

Backpropagation is performed after forward propagation. While forward propagation computes the network's output, backpropagation improves the network by updating its parameters so that future predictions become more accurate.

Definition

Backpropagation is the process of propagating the output error backward through the neural network to update the weights and biases, thereby reducing the prediction error.

Steps in Backpropagation

1. Perform forward propagation to obtain the predicted output.
2. Compare the predicted output with the actual output.
3. Calculate the prediction error.
4. Propagate the error backward through the network.
5. Update the weights and biases to reduce the error.
6. Repeat the process until the network achieves the desired accuracy.

Advantages of Backpropagation

- Reduces prediction error.
- Improves the accuracy of the neural network.
- Enables learning in multi-layer neural networks.
- Forms the basis for training most deep learning models.

11. What Makes a Neural Network "Deep"?

A neural network is called deep when it contains multiple hidden layers between the input layer and the output layer. These hidden layers enable the network to learn increasingly complex patterns from data. In a shallow neural network, there is only one hidden layer, which limits its ability to learn complex relationships. In contrast, a deep neural network contains several hidden layers, allowing it to learn hierarchical features. The initial layers learn simple features, while the deeper layers combine them to recognize more complex patterns.

Shallow Neural Network

A Shallow Neural Network contains only one hidden layer between the input and output layers. It is suitable for solving simple problems but has limited capability to learn complex patterns from large datasets.

Characteristics

- Contains a single hidden layer.
- Learns simple features from data.
- Suitable for simple classification and prediction tasks.
- Less computationally intensive.

Deep Neural Network

A Deep Neural Network (DNN) contains two or more hidden layers between the input and output layers. The additional hidden layers enable the network to learn complex features and hierarchical representations of data, making it suitable for solving challenging real-world problems.

Characteristics

- Contains multiple hidden layers.
- Learns hierarchical features automatically.
- Suitable for complex and large datasets.
- Provides higher accuracy for tasks such as image recognition, speech recognition, and natural language processing.

Hierarchical Feature Learning

One of the main advantages of deep neural networks is hierarchical feature learning. Instead of relying on manually designed features, the network automatically learns features at different levels.

For example, in image recognition:

- The first hidden layer learns simple features such as edges and lines.
- The intermediate hidden layers learn shapes and textures.
- The deeper hidden layers recognize complete objects such as faces, vehicles, or animals.

This hierarchical learning enables deep neural networks to achieve better performance on complex tasks.

12. Common Architectural Principles of Deep Networks

Deep learning models are built using several fundamental architectural principles that determine how a neural network learns and makes predictions. These principles include layers, parameters, activation functions, loss functions, optimizers, and hyperparameters, each playing a specific role in the training and performance of the network.

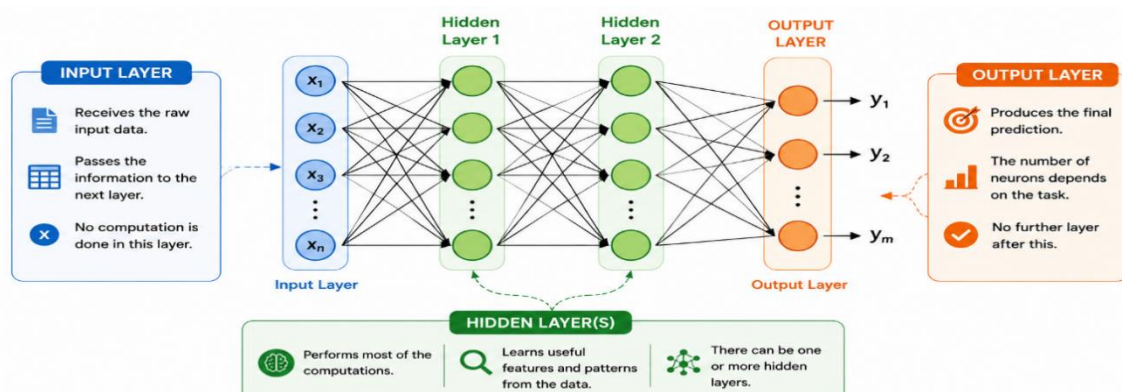
12.1 LAYERS

A neural network is organized into different layers that work together to process information. Each layer performs a specific function, such as receiving input, extracting features, or producing the final output. The arrangement of these layers enables a deep neural network to learn and solve complex problems efficiently.

Definition

A layer is a collection of interconnected artificial neurons that process information at a particular stage of a neural network. Data flows sequentially through these layers, allowing the network to learn patterns and generate predictions.

Types of Layers: A neural network consists of three main types of layers:



1. Input Layer

The Input Layer is the first layer of a neural network. It receives the input data or features from the external environment and forwards them to the hidden layer for further processing. No computations are performed in this layer.

Functions:

- Receives the input data.
- Passes the data to the hidden layer.
- Number of neurons depends on the number of input features.

2. Hidden Layer

The Hidden Layer is located between the input and output layers. It performs computations on the input data using weights, biases, and activation functions to extract meaningful features. A deep neural network contains one or more hidden layers.

Functions:

- Performs feature extraction.
- Learns patterns and relationships from data.
- Transforms inputs into useful representations.
- Improves the learning capability of the network.

3. Output Layer

The Output Layer is the final layer of a neural network. It receives the processed information from the hidden layer and produces the final prediction or decision.

Functions:

- Produces the final output.
- Performs classification or prediction.
- Number of neurons depends on the problem (e.g., one neuron for binary classification, multiple neurons for multi-class classification).

12.2 PARAMETERS

Parameters are the learnable components of a neural network. During the training process, the network automatically adjusts these parameters to minimize errors and improve prediction accuracy.

Definition

Parameters are numerical values in a neural network that are learned from the training data. They determine how input data is transformed as it passes through the network. The two main parameters are weights and biases.

Types of Parameters: A neural network has two types of parameters:

1. Weights

Weights are numerical values assigned to the connections between neurons. They determine the importance or influence of each input on the output. During training, the weights are continuously updated to improve the network's performance.

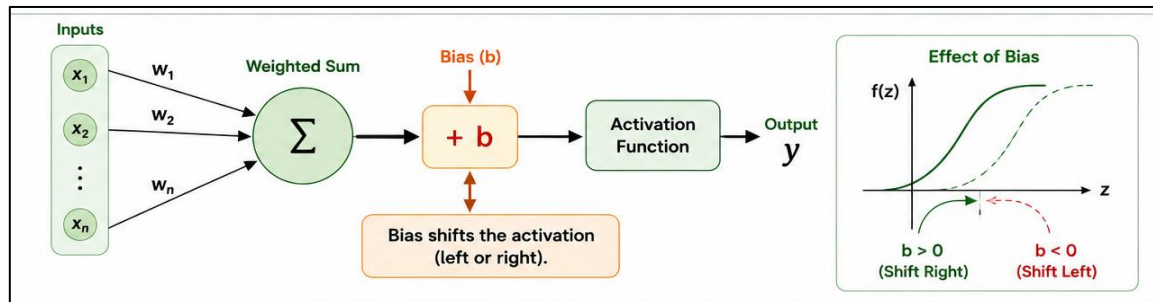
Functions:

- Determine the strength of the connection between neurons.
- Control the contribution of each input.
- Are adjusted during training to reduce prediction error.

2. Biases A bias is an additional parameter added to the weighted sum of inputs before applying the activation function. It provides flexibility to the model by allowing the activation function to shift its output, improving the network's ability to learn different patterns.

Functions:

- Shifts the activation function.
- Helps the network fit the training data more effectively.
- Improves the model's learning capability.

**Importance of Parameters**

Parameters are essential because they enable the neural network to learn from data.

- Weights determine the importance of each input feature.
- Biases provide flexibility in the learning process.
- Parameters are updated during training using the backpropagation algorithm and an optimizer.
- Well-trained parameters lead to accurate predictions and better model performance.

12.3 ACTIVATION FUNCTIONS

An activation function is an essential component of a neural network that determines the output of a neuron. It introduces non-linearity into the network, enabling deep neural networks to learn complex patterns and solve real-world problems. Without activation functions, even a network with multiple hidden layers behaves like a linear model.

Definition

An Activation Function is a mathematical function applied to the weighted sum of inputs and bias. It determines whether a neuron should be activated and passes the transformed output to the next layer.

Why Do We Need Activation Functions?

- Introduce non-linearity into the neural network.
- Enable the network to learn complex patterns and relationships.
- Help create non-linear decision boundaries.
- Improve the learning capability of deep neural networks.
- Are applied after every neuron computes its weighted sum.
- Enable the network to solve complex real-world problems.

Types of Activation Functions

The most commonly used activation functions are:

- Sigmoid Function
- Tanh (Hyperbolic Tangent) Function
- ReLU (Rectified Linear Unit)
- Leaky ReLU
- Softmax Function

1. Sigmoid Activation Function:

The Sigmoid Function maps the input value to an output between 0 and 1. Since its output can be interpreted as a probability, it is widely used in binary classification problems.

$$f(x) = \frac{1}{1 + e^{-x}}$$

Characteristics

- Output range: 0 to 1
- S-shaped (Sigmoid) curve
- Produces probability values
- Smooth and continuous function

Advantages

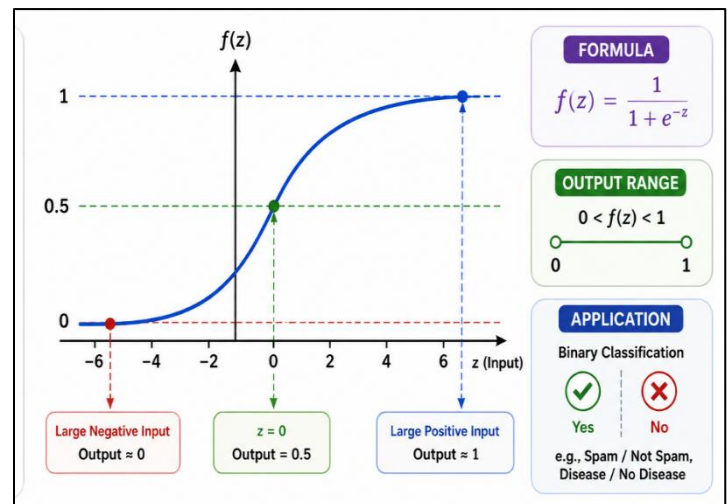
- Easy to understand and implement.
- Suitable for binary classification.
- Produces probability-like outputs.

Limitations

- Suffers from the vanishing gradient problem.
- Output is not zero-centered.
- Training becomes slower for deep networks.

Applications

- Binary classification
- Output layer of binary neural networks
- Logistic regression



2 Tanh (Hyperbolic Tangent) Activation Function

The Tanh Activation Function maps the input value to the range -1 to $+1$. It is similar to the sigmoid function but produces zero-centered outputs, which often helps the learning process.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Characteristics

- Output range: -1 to $+1$
- S-shaped curve
- Zero-centered output
- Smooth and differentiable

Advantages

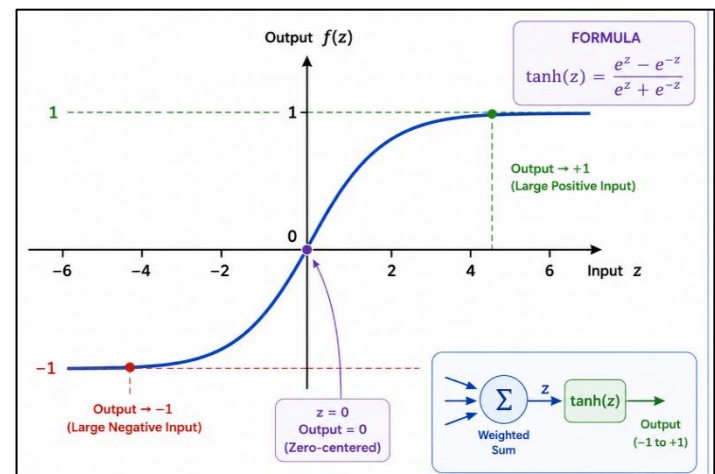
- Zero-centered output improves learning.
- Better performance than Sigmoid in many hidden layers.
- Produces both positive and negative outputs.

Limitations

- Suffers from the vanishing gradient problem.
- Less suitable for very deep networks.

Applications

- Hidden layers in neural networks
- Recurrent Neural Networks (RNNs)



3 ReLU (Rectified Linear Unit)

The ReLU (Rectified Linear Unit) activation function outputs 0 for negative input values and returns the input itself for positive values. It is the most widely used activation function in modern deep learning.

$$f(x) = \max(0, x)$$

Characteristics

- Output range: 0 to ∞
- Simple and computationally efficient

- Most widely used activation function
- Helps faster model training

Advantages

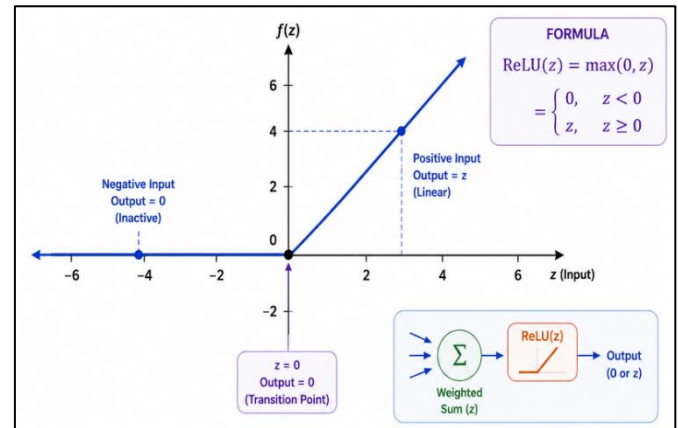
- Computationally efficient.
- Reduces the vanishing gradient problem.
- Speeds up convergence during training.
- Suitable for deep neural networks.

Limitations

- Can suffer from the dying ReLU problem, where neurons stop learning if they always output zero.

Applications

- Hidden layers of Deep Neural Networks
- Convolutional Neural Networks (CNNs)
- Most modern deep learning architectures



4 Leaky ReLU

Leaky ReLU is a modified version of the ReLU activation function that allows a small, non-zero output for negative input values. This helps reduce the dying ReLU problem.

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

Characteristics

- Allows a small negative output.
- Reduces the dying ReLU problem.
- Suitable for deep neural networks.

Applications

- Deep Neural Networks
- CNNs

5 Softmax Function

The Softmax Function converts the outputs of multiple neurons into a probability distribution whose total probability equals 1. It is commonly used in the output layer for multi-class classification problems.

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Characteristics

- Output values range from 0 to 1.
- Sum of all output probabilities equals 1.
- Produces a probability distribution over multiple classes.

Applications

- Image classification
- Handwritten digit recognition
- Text classification
- Multi-class prediction

12.4 LOSS FUNCTIONS

After a neural network generates predictions, it must determine how accurate those predictions are. This is done using a loss function, which measures the difference between the predicted output and the actual output. The loss value helps the network improve its performance during training.

Definition

A Loss Function is a mathematical function that measures the error between the predicted output and the actual output during the training of a neural network. It is also known as the Cost Function or Error Function. The objective of training is to minimize the loss value.

Need for Loss Functions

Loss functions are essential because they:

- Measure the prediction error of the model.
- Compare the predicted output with the actual output.
- Indicate how well the neural network is performing.
- Guide the optimizer to update weights and biases.
- Help improve the accuracy of the model by minimizing the error.

A lower loss value indicates better model performance, while a higher loss value indicates that the model requires further learning.

Types of Loss Functions**1. Mean Squared Error (MSE)**

Mean Squared Error (MSE) measures the average of the squared differences between the predicted values and the actual values. It is widely used for regression problems.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Characteristics

- Used for regression tasks.
- Measures the average squared prediction error.
- Larger errors receive a higher penalty.
- Lower MSE indicates better model performance.

Applications

- House price prediction
- Sales forecasting
- Temperature prediction
- Stock price prediction

2. Binary Cross-Entropy (BCE)

Binary Cross-Entropy (BCE) measures the difference between the predicted probability and the actual class in binary classification problems.

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Characteristics

- Used for binary classification.
- Works with probability values between 0 and 1.
- Commonly used with the Sigmoid activation function.
- Lower BCE indicates better predictions.

Applications

- Spam detection
- Disease prediction
- Fraud detection
- Sentiment analysis (Positive/Negative)

3. Categorical Cross-Entropy (CCE)

Categorical Cross-Entropy (CCE) measures the difference between the predicted probability distribution and the actual class in multi-class classification problems.

$$\text{CCE} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Characteristics

- Used for multi-class classification.
- Works with probability distributions.
- Commonly used with the Softmax activation function.
- Lower CCE indicates better model performance.

Applications

- Image classification
- Handwritten digit recognition
- Language identification
- Object classification

12.5 HYPERPARAMETERS

The performance of a neural network depends not only on the parameters learned during training but also on certain settings that are specified before the training process begins. These settings are called hyperparameters. Selecting appropriate hyperparameter values helps the model learn efficiently and achieve better accuracy.

Definition

Hyperparameters are user-defined settings that control the training process of a neural network. Unlike weights and biases, hyperparameters are not learned from the training data. They are chosen before training begins and influence the speed, efficiency, and performance of the model.

Importance of Hyperparameters

Hyperparameters are important because they:

- Control the learning process of the neural network.
- Affect training speed and model accuracy.
- Help improve model convergence.
- Influence the computational resources required for training.
- Need to be selected carefully through experimentation and tuning.

Common Hyperparameters

1. Learning Rate

The Learning Rate determines the size of the weight updates during training. It controls how quickly the neural network learns from the training data.

Characteristics

- Controls the speed of learning.
- Represented by the symbol η (eta).
- High learning rate may cause unstable training.
- Low learning rate makes training slower.
- Appropriate learning rate improves convergence.

2. Batch Size

Batch Size is the number of training samples processed in one iteration before updating the model parameters.

Characteristics

- Determines how many samples are processed together.
- Small batch sizes provide frequent weight updates.
- Large batch sizes improve computational efficiency.
- Larger batches require more memory.
- Common batch sizes are 16, 32, 64, and 128.

3. Number of Epochs

An Epoch is one complete pass of the entire training dataset through the neural network. The model is trained repeatedly for multiple epochs until satisfactory performance is achieved.

Characteristics

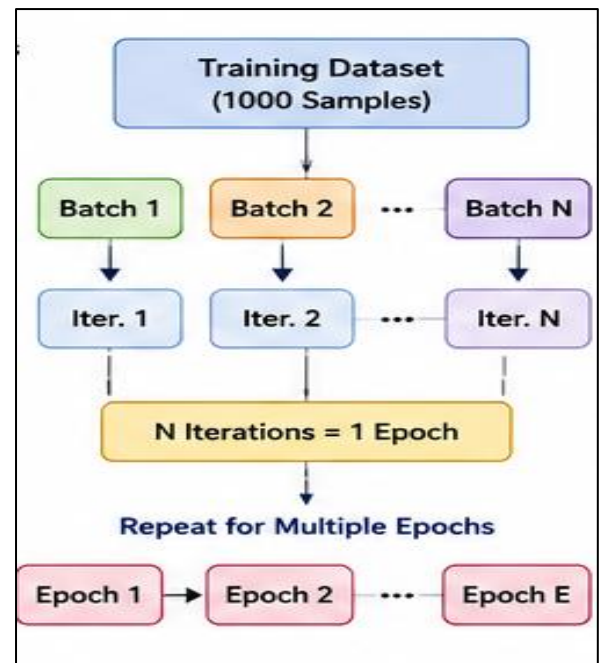
- One epoch processes the complete training dataset once.
- More epochs generally improve learning.
- Too many epochs may lead to overfitting.
- Too few epochs may result in underfitting.

4. Number of Hidden Layers

The Number of Hidden Layers determines the depth of the neural network and its ability to learn complex patterns from data.

Characteristics

- More hidden layers increase learning capacity.
- Deep networks can learn hierarchical features.
- Too many layers increase computational complexity.
- The appropriate number depends on the problem being solved.



Hyperparameter	If Too Low	If Optimal	If Too High
 Learning Rate	Slow learning Convergence takes more time	Faster convergence Stable training	Unstable training May overshoot the minimum
 Batch Size	Too small: Slow training Frequent updates	Balanced speed and memory utilization	Too large: High memory usage
 Epochs	Too few: Underfitting (Model does not learn well)	Enough to learn patterns in data	Too many: Overfitting (Model learns noise)
 Hidden Layers / Neurons	Too few: Low learning capacity	Good balance: Better accuracy	Too many: High complexity, Overfitting risk

13. Building Blocks of Deep Networks

Deep learning has evolved through several neural network architectures that introduced new learning capabilities and solved the limitations of earlier models. These architectures serve as the building blocks of deep networks and have played an important role in the development of modern deep learning systems. In this unit, we focus on two foundational architectures: Restricted Boltzmann Machines (RBMs) and Autoencoders.

13.1 Restricted Boltzmann Machine (RBM)

Deep neural networks require efficient methods to learn useful features from large amounts of unlabeled data. One such method is the Restricted Boltzmann Machine (RBM), which automatically discovers hidden patterns and meaningful representations without requiring labeled examples. RBMs became an important milestone in the evolution of deep learning and served as the foundation for Deep Belief Networks (DBNs).

Definition

A Restricted Boltzmann Machine (RBM) is an unsupervised generative neural network that learns useful features and hidden patterns from unlabeled data. Instead of predicting an output, an RBM automatically learns meaningful representations of the input data.

Key Characteristics

An RBM has the following characteristics:

- It is an unsupervised learning model.
- It is a generative neural network.
- Learns hidden features from unlabeled data.
- Performs automatic feature learning.
- Forms the foundation for Deep Belief Networks (DBNs).

Architecture of Restricted Boltzmann Machine

An RBM consists of two layers of neurons that communicate with each other to learn hidden features from the input data.

1. Visible Layer (Input Layer)

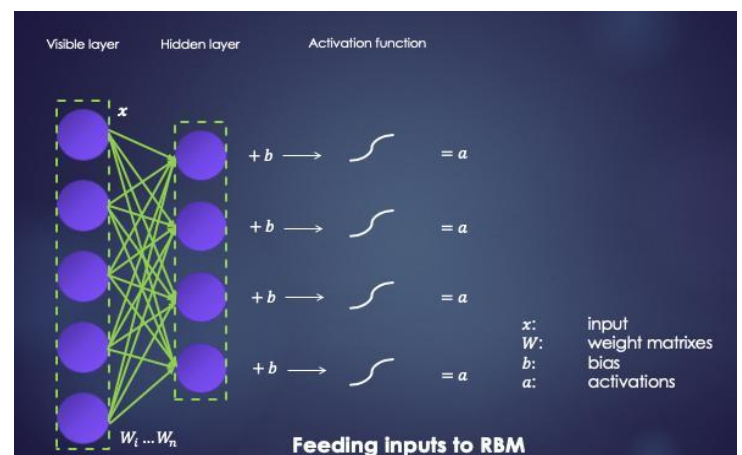
- Receives the input data.
- Each neuron represents one feature of the input.
- Also called the Input Layer.

2. Hidden Layer

- Learns meaningful features and hidden patterns.
- Represents the internal knowledge learned from the input data.

3. Connections

- Every visible neuron is connected to every hidden neuron.
- There are no connections between neurons within the same layer.
- This restriction gives the model its name Restricted Boltzmann Machine.



Working of Restricted Boltzmann Machine

The working of an RBM involves the following steps:

Step	Description
Input Data	The visible layer receives the input data.
Feature Learning	The hidden layer learns meaningful features and patterns from the input.
Reconstruction	The hidden layer reconstructs the input back to the visible layer.
Error Calculation	The reconstructed data is compared with the original input to measure the reconstruction error.
Weight Update	The connection weights are adjusted to reduce the reconstruction error.
Repeat	The process continues until the RBM learns an accurate representation of the data.

Advantages

- Learns from unlabeled data without requiring class labels.
- Automatically extracts meaningful hidden features.
- Reduces the need for manual feature engineering.
- Useful for feature learning and dimensionality reduction.
- Forms the foundation for Deep Belief Networks (DBNs).

Limitations

- Training is computationally expensive.
- Training becomes slower for large datasets.
- Sensitive to hyperparameter selection.
- Reconstruction may not always produce accurate results.
- Less commonly used in modern deep learning compared to Autoencoders and other architectures.

Applications

- Recommendation Systems
- Feature Extraction
- Pattern Recognition
- Data Representation

13.2 Autoencoder

In many real-world applications, it is useful to learn a compact representation of data while preserving its important information. An Autoencoder is a deep learning model that automatically compresses the input data into a lower-dimensional representation and then reconstructs the original data. It is widely used for feature learning, dimensionality reduction, and anomaly detection.

Definition

An Autoencoder is an unsupervised neural network that learns an efficient, compressed representation of input data and reconstructs the original data from it.

Key Characteristics

An Autoencoder has the following characteristics:

- It is an unsupervised learning model.
- Learns compressed (latent) representations of data.
- Performs automatic feature learning.
- Reconstructs the original input from the learned representation.
- Widely used for dimensionality reduction and anomaly detection.

Architecture of Autoencoder

An Autoencoder consists of three main components that work together to compress and reconstruct the input data.

1. Encoder

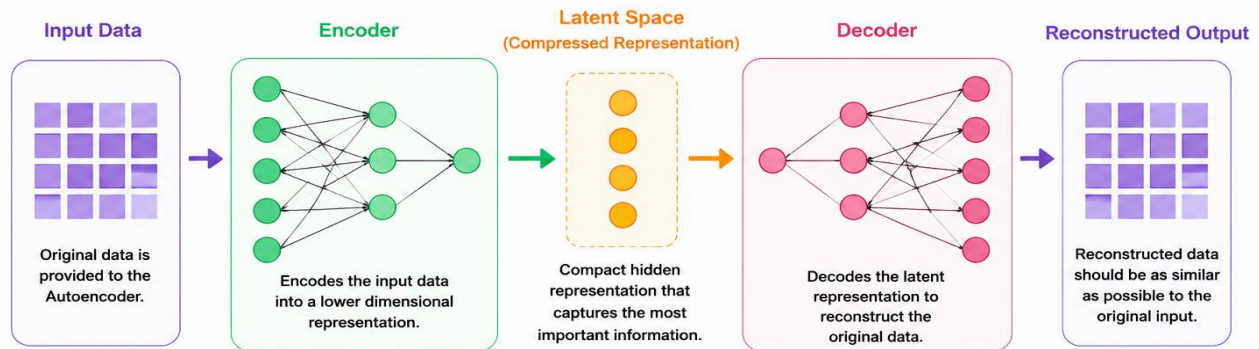
- Receives the input data.
- Compresses the input into a lower-dimensional representation.
- Extracts the most important features from the input.

2. Latent Space (Compressed Representation)

- Stores the compressed representation of the input.
- Contains the essential information required to reconstruct the original data.
- Also called the Bottleneck Layer.

3. Decoder

- Receives the compressed representation.
- Reconstructs the original input data.
- Produces an output that is as close as possible to the original input.



Working of Autoencoder

The working of an Autoencoder involves the following steps:

Step	Description
Input Data	The Autoencoder receives the original input data.
Encoding	The encoder compresses the input into a lower-dimensional representation.
Latent Representation	The compressed representation stores the most important features of the input.
Decoding	The decoder reconstructs the original data from the latent representation.
Reconstruction Error	The reconstructed output is compared with the original input to measure the error.
Parameter Update	The network updates its weights to minimize the reconstruction error and improve future reconstructions.

Advantages

- Learns from unlabeled data.
- Automatically extracts meaningful features.
- Performs efficient dimensionality reduction.
- Compresses data effectively.
- Useful for anomaly detection and data reconstruction.

Limitations

- May reconstruct noisy or irrelevant features.
- Requires sufficient training data for good results.
- Can overfit if the model is too complex.
- Training may be computationally expensive for large datasets.
- Not suitable for all types of learning problems.

Applications

- Dimensionality Reduction
- Feature Extraction
- Anomaly Detection
- Image Denoising
- Data Compression
- Image Reconstruction